

Міністерство освіти і науки України
Донецький національний університет імені Василя Стуса
Факультет інформаційних і прикладних технологій

Кафедра інформаційних технологій

ЗВІТ

З лабораторної роботи №1
дисципліни «Інтелектуальний аналіз даних та штучний інтелект»
на тему: «Синтез оптимальної регресійної моделі за експериментальними
даними»

Виконав: Молодченко Д. В.,
студент 4 курсу спеціальності
122 КН , групи Б

Перевірив: Штовба С. Д.,
професор кафедри
інформаційних технологій

Вінниця 2025

Варіант 42

Датасет

Лабораторна робота передбачає використання експериментальних даних з набору даних *Residential Building Data Set* з репозиторію UCI Machine Learning Repository. Коден рядок таблиці відповідає одному житловому об'єкту – переважно багатоквартирним будівлям 3-9 поверхів збудованим у Тегерані в проміжку приблизно 1993-2008 роки. В даному датасеті міститься 372 об'єкти, кожен з яких має 109 стовпців серед яких:

1. Календарні характеристики проекту;
2. 8 фізичних та фінансових змінних;
3. 19 економічних показників, кожен у 5 часових лагах (часовий лаг -);
4. 2 вихідні (цільові) змінні;

Всі змінні є числовими, відсутні пропущені значення, що спрощує попередню обробку побудову моделей.

Календарні змінні

START YEAR, START QUARTER – рік та квартал початку будівництва.

COMPLETION YEAR, COMPLETION QUARTER – рік та квартал завершення будівництва.

Ці змінні не будуть використовуватись у лабораторній роботі.

Фізичні та фінансові змінні

V-1 – локація проекту у форматі поштового індексу району міста;

V-2 – загальна площа квартир у будинку у m^2 ;

V-3 – площа земельної ділянки у m^2 ;

V-4 – загальна попередня оцінка вартості будівництва за цінами на початок проекту, масштаб 10 000 000 IRRm (Monthly Internal Rate of Return – показує який відсоток прибутку інвестор отримує щомісяця від вкладених коштів у конкретний житловий проект);

V-5 – попередньо оцінена вартість будівництва, масштаб 10 000 IRRm. Одна з вхідних змінних даного варіанту;

V-6 – еквівалентна попередня вартість будівництва перерахована в цінах деякого базового року;

V-7 – тривалість будівництва в місяцях;

V-8 – ціна за 1 m^2 житла на початку проекту, масштаб 10 000 IRRm;

Вихідні змінні

V-9 – фактична ціна продажу об'єкта, масштаб 10 000 IRRm. Цільова змінна для даного варіанту;

V-10 – фактична собівартість будівництва, масштаб 10 000 IRRm;

Економічні змінні та індекси

Це макроекономічні показники пов'язані з ринком будівництва, кредитування, валютним курсом та загальним станом економіки. Для кожного з них у датасеті наведено значення у п'яти часових лагах відносно старту проекту. Такий підхід дозволяє враховувати вплив економічної ситуації за декілька періодів до початку будівництва. У таблиці такі лаги реалізовано як V-XX, V-XX.1, V-XX.2, V-XX.3, V-XX.4.

V-11 – кількість виданих дозволів на будівництво;

V-12 – індекс будівельних послуг;

V-13 – індекс оптових цін на будівельні матеріали. Одна з вхідних змінних у даному варіанті;

V-14 – сумарна площа будинків у виданих дозволах у м²;

V-15 – кумулятивна ліквідність у економіці, масштаб 10 000 000 IRRm;

V-16 – інвестиції приватного сектору в нове житлове будівництво, масштаб 10 000 000 IRRm;

V-17 – індекс цін на землю у базовому році.

V-18 – кількість банківських кредитів виданих за певний період;

V-19 – сумарний обсяг виданих кредитів, масштаб 10 000 000 IRRm;

V-20 – відсоткова ставка за кредитами у %;

V-21, V-22 – середня вартість будівництва будинків приватного сектору на початку та в кінці будівництва, масштаб 10 000 IRRm/м²;

V-23, V-24 – офіційний та ринковий курс національної валюти до долара США;

V-25, V-26 – загальний індекс споживчих цін (CPI – Consumer Price Index) та CPI для житла/води/паливно-енергетичного сектору;

V-27 – фондовий індекс;

V-28 – чисельність населення міста;

V-29 – ціна золота за унцію, IRRm;

Постановка задачі

Синтезувати оптимальну двофакторну регресійну модель виду $y = f(x_1, x_2)$. Потрібно за експериментальними даними відновити залежність фактичної ціни продажу житлового будинку від попередньо оціненої вартості будівництва та індексу оптових цін на будівельні матеріали.

У даній лабораторній роботі розглядається задача побудови та аналізу оптимальної двофакторної регресійної моделі, яка відновлює залежність фактичної ціни продажу житлового об'єкта від вхідних змінних згідно з варіантом 42.

Вхідні змінні

V-5 – попередньо оцінена вартість будівництва.

V-13 – індекс оптових цін на будівельні матеріали у базовому році.

Вихідна змінна

V-9 – фактична ціна продажу житлового об'єкта.

Мета: відновити двофакторну регресійну залежність для задачі.

Підготовка даних:

```
FILE = '/MATLAB Drive/Residential-Building-Data-Set.xlsx';
T = readtable(FILE, 'Sheet', 'Data', 'NumHeaderLines', 1);

x1 = T.V_5; x1 = x1(:);
x2 = T.V_13; x2 = x2(:);
y = T.V_9; y = y(:);

N = height(T);
```

Лістинг 1. Зчитування та підготовка базових даних датасету.

Даний лістинг зчитує потрібні дані з таблиці та ініціалізує основні змінні для подальшої роботи.

Завдання 2

Розбити дані на навчальну та тренувальну вибірки та перевірити їх репрезентативність. Нехай поділ даних матиме таку пропорцію 70/30.

```
[trainIdx, testIdx] = ida_holdout_split(N, 0.7, 1);
x_train = [x1(trainIdx), x2(trainIdx)];
y_train = y(trainIdx);
x_test = [x1(testIdx), x2(testIdx)];
y_test = y(testIdx);
S = ida_split_stats([x1, x2, y], trainIdx, testIdx, ["V5", "V13", "V9"]);
disp(S);
```

Лістинг 2. Розподіл повної вибірки на навчальну та тестову за допомогою ida_holdout_split та виведення результатів через ida_split_stats.

```
function [trainIdx, testIdx] = ida_holdout_split(N, trainRatio, seed)
rng(seed);
idx = randperm(N);
Ntrain = round(trainRatio * N);
trainIdx = idx(1:Ntrain);
testIdx = idx(Ntrain+1:end);
end
```

Лістинг 3. Функція розподілу на train та test.

```
function S = ida_split_stats(A, trainIdx, testIdx, varNames)
A = double(A);
Atr = A(trainIdx, :);
Ate = A(testIdx, :);

S = table(string(varNames(:)), ...
    mean(A, 1, 'omitnan'), std(A, 0, 1, 'omitnan'), min(A, [], 1, 'omitnan'), max(A,
[], 1, 'omitnan'), ...
    mean(Atr, 1, 'omitnan'), std(Atr, 0, 1, 'omitnan'), min(Atr, [], 1, 'omitnan'),
max(Atr, [], 1, 'omitnan'), ...
    mean(Ate, 1, 'omitnan'), std(Ate, 0, 1, 'omitnan'), min(Ate, [], 1, 'omitnan'),
max(Ate, [], 1, 'omitnan'), ...
    'VariableNames',
    {'Var', 'Mean_All', 'Std_All', 'Min_All', 'Max_All', 'Mean_Train', 'Std_Train', 'Min_Train', 'M
ax_Train', 'Mean_Test', 'Std_Test', 'Min_Test', 'Max_Test'} );
end
```

Лістинг 4. Функція виведення статистики розподілу.

Що робить код розподілу: він створює послідовність індексів у випадковому порядку. Ця послідовність потім розділяється на дві послідовності: індекси навчальних даних та тестових відповідно до заданої пропорції 70/30. Перевірка репрезентативності буде здійснена шляхом порівняння наступних статистик: середнє значення, стандартне відхилення, мінімум та максимум.

Статистика для x1 (V5)			
	All	Train	Test
	-----	-----	-----
Mean	163.12	168	151.79
Std	112.6	116.18	103.44
Min	10	10	10
Max	640	640	400

Рисунок 1. Статистики для попередньо оціненої вартості будівництва у повній, навчальній та тестовій вибірках.

Статистика для x2 (V13)			
	All	Train	Test
	-----	-----	-----
Mean	88.05	90.639	82.043
Std	49.364	49.758	48.121
Min	10.03	10.03	10.03
Max	225	225	225

Рисунок 2. Статистики для економічного індексу у повній, навчальній та тестовій вибірках.

Статистика для y (V9)			
	All	Train	Test
Mean	1387.4	1422.9	1305.2
Std	1206.1	1226.3	1159
Min	50	130	50
Max	6800	6800	5100

Рисунок 3. Статистики для фактичної ціни продажу у повній, навчальній та тестовій вибірках.

Отримано таблицю для порівняння повної, тренувальної та тестової вибірок. З неї можна побачити, що середні значення та відхилення в навчальній та тестовій вибірках є дуже близькими до відповідних значень у повній вибірці. Не спостерігається сильного зсуву в бік малих чи великих значень, тому можна сказати, що діапазон значень нових вибірок добре покриває діапазон повної вибірки. Розмір вибірок: навчальна – 260, тестова – 112 об’єктів. Такий розмір є достатнім.

```
ida_plot_split_scatter(x_train(:,1), y_train, x_test(:,1), y_test, ...
    'V5', 'V9', 'Train/Test: V9 vs V5');

ida_plot_split_scatter(x_train(:,2), y_train, x_test(:,2), y_test, ...
    'V13', 'V9', 'Train/Test: V9 vs V13');
```

Лістинг 5. Візуалізація розподілених даних.

```
function ida_plot_split_scatter(x_tr, y_tr, x_te, y_te, xlabel, ylabel, ttl)
figure;
hold on;
scatter(x_tr, y_tr, 20, 'magenta', 'filled');
scatter(x_te, y_te, 20, 'cyan', 'filled');
hold off;
grid on;
xlabel(xlabel, 'Interpreter', 'none');
ylabel(ylabel, 'Interpreter', 'none');
title(ttl, 'Interpreter', 'none');
legend('Train', 'Test', 'Location', 'best');
set(gca, 'FontName', 'Times New Roman', 'FontSize', 12);
end
```

Лістинг 6. Функція візуалізації розподілених даних.

Даний лістинг бере окремо x_1 та x_2 з навчальної та тестової вибірок та створює графік залежності V-9 від кодної з них.

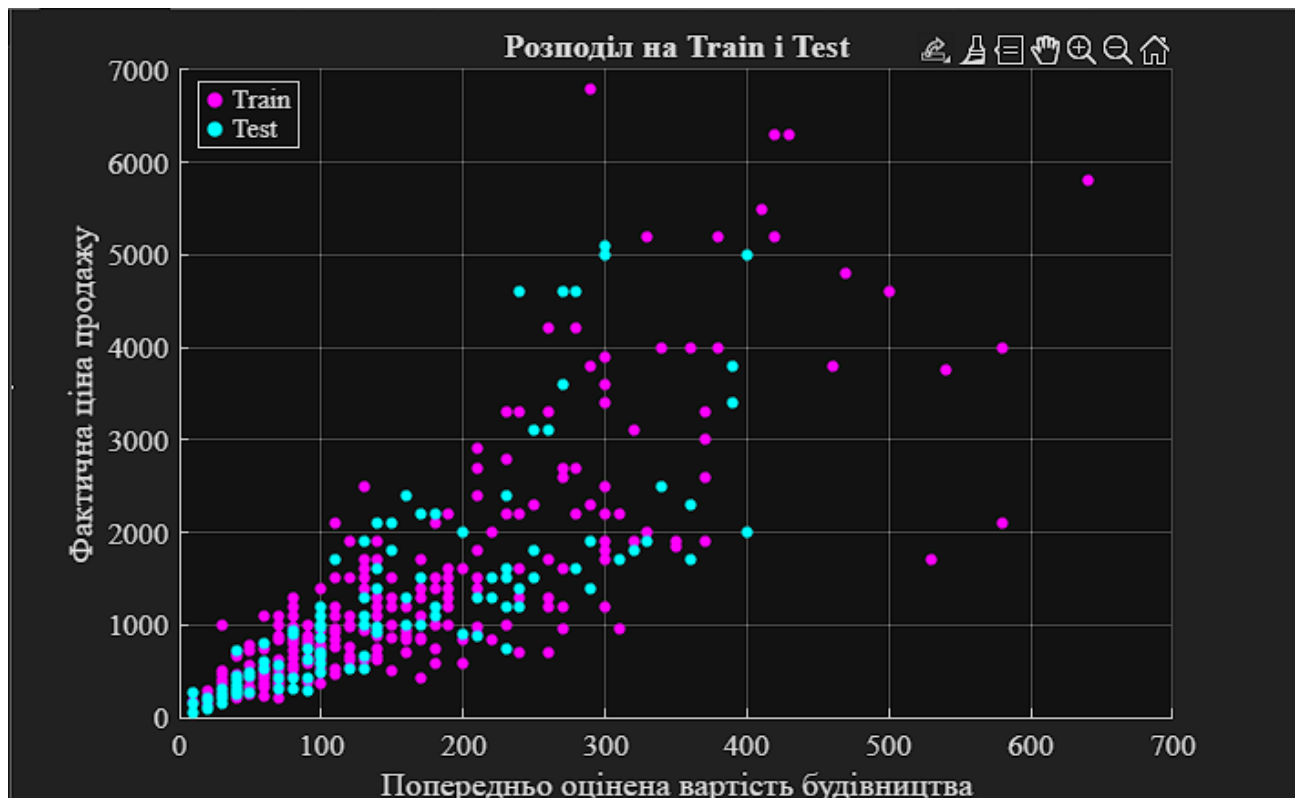


Рисунок 4. Розподіл на навчальну та тестову вибірки. Залежність V-9 від V-5.

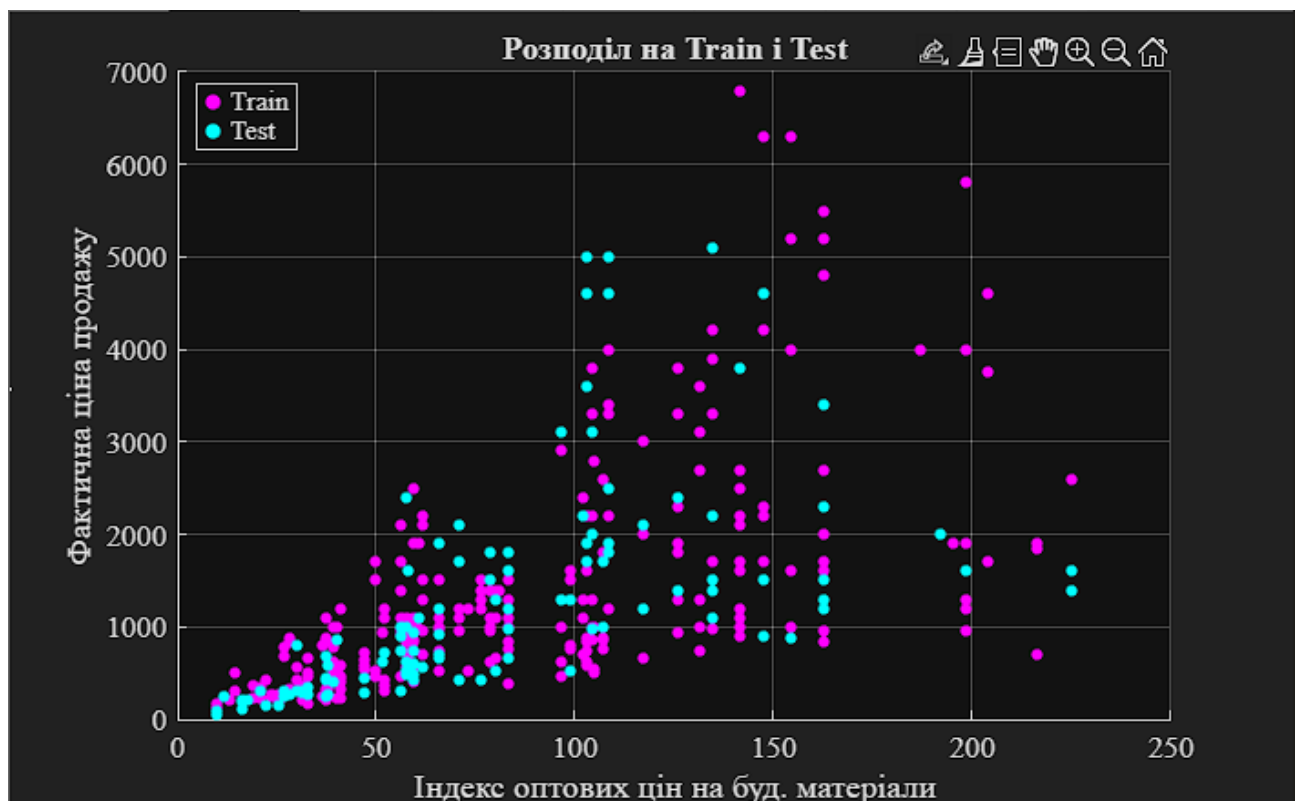


Рисунок 5. Розподіл на навчальну та тестову вибірки. Залежність V-9 від V-13.

Висновок по репрезентативності: на підставі вище наведених числових даних можна підсумувати, що навчальна та тестова вибірки є репрезентативними.

Завдання 3

Зобразити експериментальні дані у формі однофакторних залежностей

```
if ~exist('T','var') || ~exist('x1','var')
    ida_ll_data;
end

ida_plot_scatter(x1, y, 'V5', 'V9', 'V9 vs V5');
ida_plot_scatter(x2, y, 'V13', 'V9', 'V9 vs V13');
```

Лістинг 7. Візуалізація залежностей між вхідними змінними та вихідною змінною.

```
function ida_plot_scatter(x, y, xlabel, ylabel, ttl)
figure;
scatter(x, y, 'filled');
grid on;
xlabel(xlabel, 'Interpreter','none');
ylabel(ylabel, 'Interpreter','none');
title(ttl, 'Interpreter','none');
set(gca, 'FontName', 'Times New Roman', 'FontSize', 12);
end
```

Лістинг 8. Функція візуалізація залежностей.

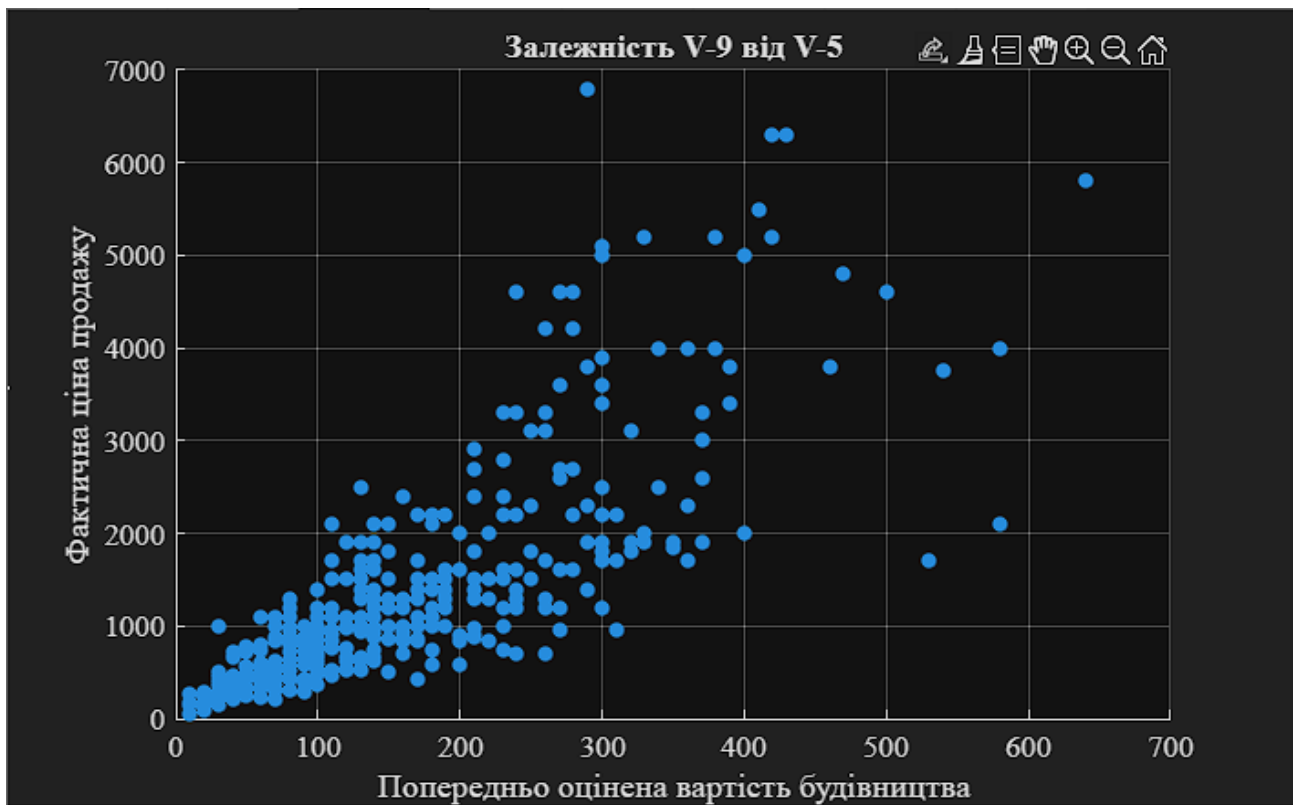


Рисунок 6. Однофакторна залежність фактичної ціни продажу від попередньо оціненої вартості будівництва.

Спостерігається висхідна тенденція. Зі збільшенням попередньої оцінки вартості будівництва V-5 зростає фактична ціна продажу V-9. Досить щільна концентрація точок навколо уявної висхідної лінії свідчить про сильний позитивний зв'язок між V-5 та V-9. Для однакових значень V-5 є певний розкид значень V-9 – це пов'язано із впливом інших факторів.

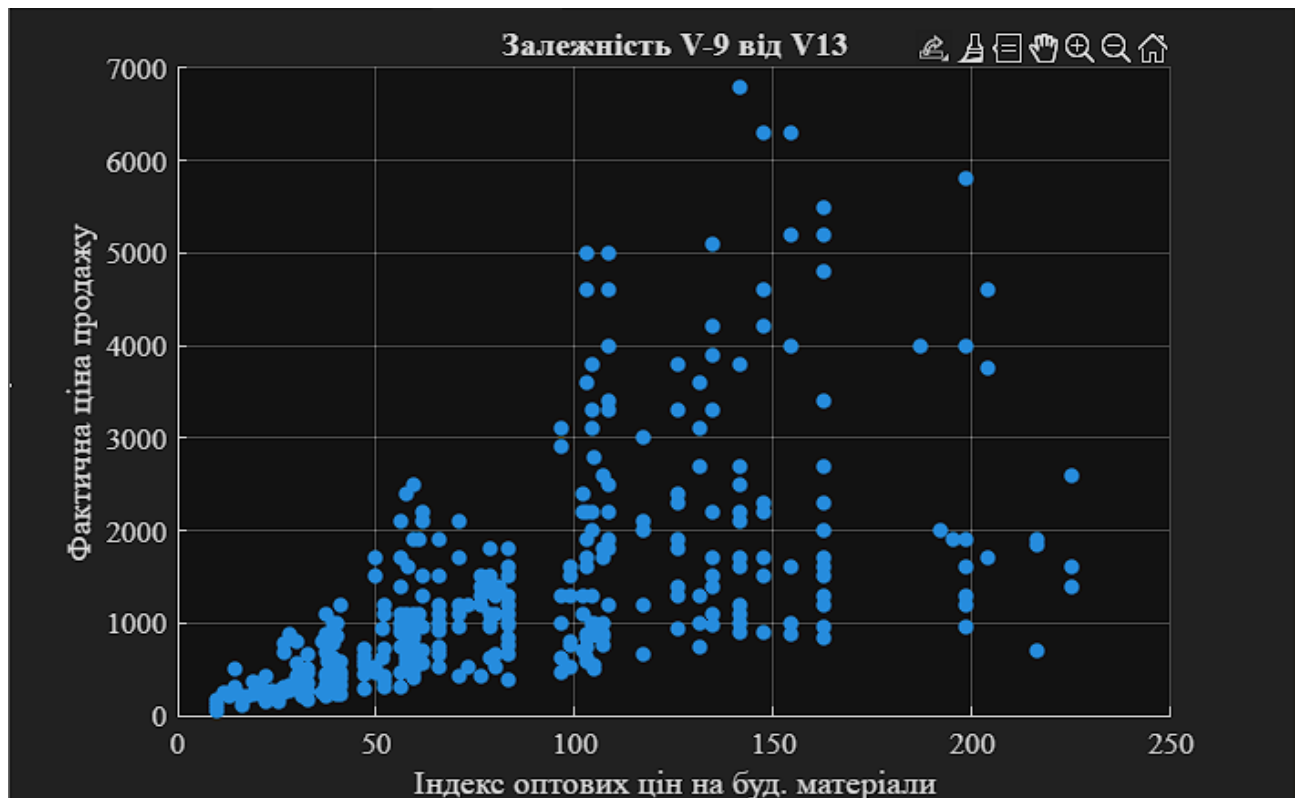


Рисунок 7. Однофакторна залежність фактичної ціни продажу від економічного індексу.

Даний графік також зображає позитивну тенденцію, яка говорить, що при збільшенні економічного індексу V-13 в середньому зростає і фактична ціна продажу V-9. Розкид точок навколо уявної трендової лінії є більшим ніж у випадку з V-5, тому можна зробити висновок про слабший зв'язок. Тож вплив V-13 на V-9 є помітним але слабшим ніж V-5 на V-9.

Отже обидва фактори позитивно впливають на фактичну ціну продажу. Вплив V5 є сильнішим в порівнянні з впливом V13.

Завдання 4

Віднайти коефіцієнти п'яти регресійних залежностей, серед яких дві моделі слід згенерувати самостійно.

Лінійна модель (M1): $y = a_0 + a_1x_1 + a_2x_2$;

Квадратична без взаємодії (M2): $y = a_0 + a_1x_1 + a_2x_2 + a_3x_1^2 + a_4x_2^2$;

Повна квадратична з взаємодією (M3):

$$y = a_0 + a_1x_1 + a_2x_2 + a_3x_1^2 + a_4x_2^2 + a_5x_1x_2;$$

Кубічна без змішаних кубічних членів (M4):

$$y = a_0 + a_1x_1 + a_2x_2 + a_3x_1^3 + a_4x_2^3 + a_5x_1^2 + a_6x_2^2;$$

Змішана кубічна з взаємодіями вищого порядку (M5):

$$y = a_0 + a_1x_1 + a_2x_2 + a_3x_1x_2 + a_4x_1^2x_2 + a_5x_1x_2^2;$$

```

if ~exist('T','var') || ~exist('x_train','var')
    ida_ll_data;
end

t4 = ida_fit_models2(x_train, y_train, x_test, y_test);
disp(t4.results);

for m = 1:5
    disp("Коефіцієнти " + t4.modelNames(m) + ":");
    disp(t4.a{m});
end

```

Лістинг 9. Пошук коефіцієнтів моделей.

```

function out = ida_fit_models2(x_train, y_train, x_test, y_test)
x1_tr = x_train(:,1); x2_tr = x_train(:,2); y_tr = y_train(:);
x1_te = x_test(:,1); x2_te = x_test(:,2); y_te = y_test(:);
mtr = isfinite(y_tr) & isfinite(x1_tr) & isfinite(x2_tr);
mte = isfinite(y_te) & isfinite(x1_te) & isfinite(x2_te);
x1_tr = x1_tr(mtr); x2_tr = x2_tr(mtr); y_tr = y_tr(mtr);
x1_te = x1_te(mte); x2_te = x2_te(mte); y_te = y_te(mte);
modelNames = ["M1 linear", "M2 quad", "M3 full quad", "M4 cubic", "M5 mixed cubic"];
p = zeros(5,1);
RMSE_Train = zeros(5,1); R2_Train = zeros(5,1);
RMSE_Test = zeros(5,1); R2_Test = zeros(5,1);
a = cell(5,1);
yhat_tr = cell(5,1);
yhat_te = cell(5,1);

for m = 1:5
    Xtr = ida_designMatrix2(m, x1_tr, x2_tr);
    Xte = ida_designMatrix2(m, x1_te, x2_te);
    a{m} = lsqminnorm(Xtr, y_tr);
    yhat_tr{m} = Xtr * a{m};
    yhat_te{m} = Xte * a{m};
    p(m) = size(Xtr,2);
    [RMSE_Train(m), R2_Train(m)] = ida_metrics(y_tr, yhat_tr{m});
    [RMSE_Test(m), R2_Test(m)] = ida_metrics(y_te, yhat_te{m});
end

results = table((1:5)', modelNames', p, RMSE_Train, R2_Train, RMSE_Test, R2_Test, ...
    'VariableNames',
    {'Model','Name','p','RMSE_Train','R2_Train','RMSE_Test','R2_Test'});
out.modelNames = modelNames;
out.a = a;
out.yhat_tr = yhat_tr;
out.yhat_te = yhat_te;
out.results = results;
out.y_tr = y_tr;
out.y_te = y_te;
end

```

Лістинг 10. Функція пошуку коефіцієнтів та навчання моделей.

Даний скрипт будує п'ять регресійних моделей, знаходить для кожної з них коефіцієнти, обчислює прогнозовані значення для навчальної та тестової вибірок та оцінює якість кожної моделі за RMSE та R^2 .

```
Коефіцієнти моделі 1
76.5323
8.8677
-1.5826
```

Рисунок 8. Коефіцієнти моделі 1

```
Коефіцієнти моделі 2
-326.1136
7.7579
10.5281
0.0020
-0.0556
```

Рисунок 9. Коефіцієнти моделі 2

```
Коефіцієнти моделі 3
-300.7904
5.3016
15.2416
-0.0112
-0.1647
0.0847
```

Рисунок 10. Коефіцієнти моделі 3

```
Коефіцієнти моделі 4
527.1687
-6.4168
0.7749
0.0618
0.1029
-0.0001
-0.0006
```

Рисунок 11. Коефіцієнти моделі 4

```
Коефіцієнти моделі 5
939.7225
-6.6464
-12.7478
0.2530
-0.0000
-0.0008
```

Рисунок 12. Коефіцієнти моделі 5

Model	p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
1	3	750.35	0.62414	729.92	0.59982
2	5	737.81	0.6366	715.49	0.61548
3	6	694.43	0.67808	744.99	0.58312
4	7	695.5	0.67708	753	0.5741
5	6	669.33	0.70093	747.72	0.58006

Рисунок 13. Коефіцієнти RMSE та R^2 для п'яти моделей

Отже під час виконання даного завдання було побудовано 5 регресійних моделей для відновлення залежності V-9 від V-5 та V-13 та знайдено для них коефіцієнти

за методом найменших квадратів. Також було розраховано показники точності на навчальній та тестовій вибірках. Отримані дані показують покращення точності для навчальних даних зі збільшенням складності моделі, але для тестової вибірки результат є зовсім іншим. На тестових даних найкраще справилася квадратична модель (Модель 2). Суттєве погіршення якості на тестовій вибірці у складніших моделях свідчить про перенавчання.

Завдання 5

Орієнтуючись на результати, квадратична модель дає кращу узагальнювальну здатність. На тестових даних різниця в RMSE дорівнює 14.43 одиниці, що становить ~2%. Коефіцієнт детермінації R^2 також є більшим у квадратичній моделі. Це говорить про кращий опис фактичних значень V-9 на тестовій вибірці.

```
if ~exist('T','var') || ~exist('x_train','var')
    ida_ll_data;
end
if ~exist('t4','var')
    t4 = ida_fit_models2(x_train, y_train, x_test, y_test);
end

ida_plot_yhat(t4.y_tr, t4.yhat_tr{1}, 'Лінійна (Train)');
ida_plot_yhat(t4.y_te, t4.yhat_te{1}, 'Лінійна (Test)');

ida_plot_yhat(t4.y_tr, t4.yhat_tr{2}, 'Квадратична (Train)');
ida_plot_yhat(t4.y_te, t4.yhat_te{2}, 'Квадратична (Test)');
    Лістинг 11. Візуалізація train vs test для лінійної та квадратичної моделей.
```

```
function ida_plot_yhat(y_actual, y_pred, ttl)
figure;
scatter(y_actual, y_pred, 16, 'filled');
grid on; hold on;

mn = min([y_actual(:); y_pred(:)]);
mx = max([y_actual(:); y_pred(:)]);
plot([mn mx], [mn mx], '--', 'LineWidth', 1, 'Color', 'w');

hold off;

[rmse, r2] = ida_metrics(y_actual, y_pred);

xlabel('Фактичні', 'Interpreter','none');
ylabel('Прогнозовані', 'Interpreter','none');
title(sprintf('%s | RMSE=%.4f | R2=%.4f', ttl, rmse, r2), 'Interpreter','none');
end
```

Лістинг 12. Функція візуалізації train vs test.

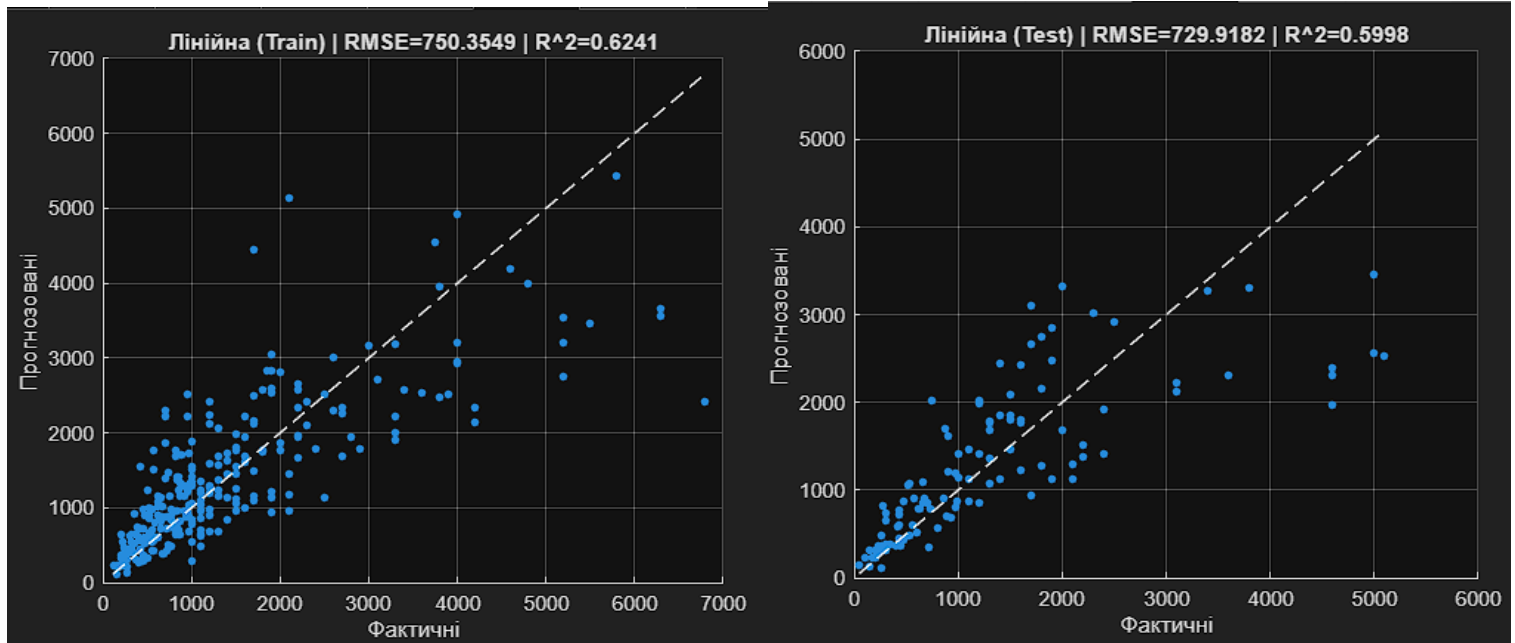


Рисунок 14. Візуалізація прогнозів лінійної моделі.

Якщо додатково опиратися на графіки фактичних значень проти прогнозованих то для лінійної моделі видно, що точки часто відходять від прямої $y = \hat{y}$ особливо у області більших фактичних значень. Також можна помітити що розкид точок ніяко не діагоналі досить широкий, тобто дисперсія похибок є доволі відчутною.

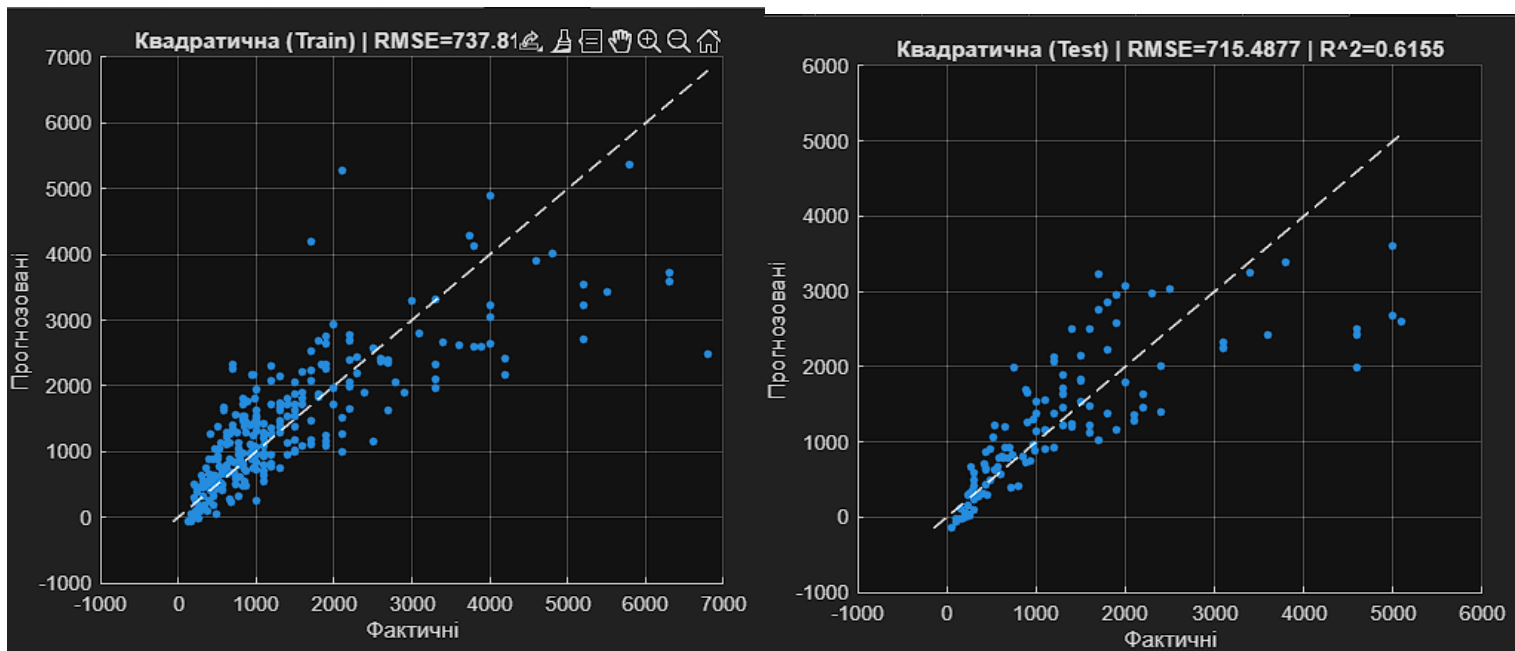


Рисунок 15. Візуалізація прогнозів квадратичної моделі.

Для квадратичної моделі на аналогічних графіках точки в середньому розташовані ближче до діагоналі. Точки є більш зібраними. У верхній частині діапазону фактичних значень відхилення квадратичної моделі є меншими ніж у лінійної, але розкид всеодно зберігається. Очевидно, що це говорить про вплив інших факторів.

Вибір оптимальної моделі:

Між лінійною і квадратичною моделями оптимальною було обрано квадратичну (Модель 2) за сукупністю кращих критеріїв точності, помірного ускладнення та збереження достатньої інтерпретабельності.

Завдання 6

Почергово додати у постановку задачі ще одну вхідну змінну, і побудувати усі можливі лінійні регресійні моделі з трьома змінними та порівняти їх з моделлю з попереднього завдання.

```
if ~exist('T','var') || ~exist('trainIdx','var')
    ida_l1_data;
end

t6 = ida_task6_scan(T, trainIdx, testIdx);

disp('TOP-10 за RMSE_Test:');
disp(t6.t6_sorted(1:min(10,height(t6.t6_sorted)), :));

ida_plot_yhat(t6.best.ytr, t6.best.yhat_tr, t6.best.VnName + ", " + t6.best.ModelName +
" (Train)");
ida_plot_yhat(t6.best.yte, t6.best.yhat_te, t6.best.VnName + ", " + t6.best.ModelName +
" (Test)");
```

Лістинг 13. Пошук найкращої комбінації вхідних змінних.

```
function out = ida_task6_scan(T, trainIdx, testIdx)
vars = T.Properties.VariableNames;
x1 = T.V_5; x1 = x1(:);
x2 = T.V_13; x2 = x2(:);
y = T.V_9; y = y(:);
isNum = varfun(@isnumeric, T, 'OutputFormat','uniform');
cand = vars(isNum);
cand = setdiff(cand, {'V_5','V_13','V_9'}, 'stable');
modelName = ["M1 linear", "M2 quad", "M3 full quad", "M4 cubic", "M5 mixed cubic"];

rows = numel(cand) * 5;
VnName = strings(rows,1);
ModelId = zeros(rows,1);
ModelName = strings(rows,1);
p = zeros(rows,1);
RMSE_Train = zeros(rows,1);
R2_Train = zeros(rows,1);
RMSE_Test = zeros(rows,1);
R2_Test = zeros(rows,1);

bestRMSE = inf;
best = struct();

r = 0;

for i = 1:numel(cand)
    x3 = T{:, cand{i}};
    x3 = x3(:);
```

```

    x1_tr = x1(trainIdx); x2_tr = x2(trainIdx); x3_tr = x3(trainIdx); y_tr =
y(trainIdx);
    x1_te = x1(testIdx); x2_te = x2(testIdx); x3_te = x3(testIdx); y_te =
y(testIdx);

    x1_tr = x1_tr(:); x2_tr = x2_tr(:); x3_tr = x3_tr(:); y_tr = y_tr(:);
    x1_te = x1_te(:); x2_te = x2_te(:); x3_te = x3_te(:); y_te = y_te(:);

    mtr = isfinite(y_tr) & isfinite(x1_tr) & isfinite(x2_tr) & isfinite(x3_tr);
    mte = isfinite(y_te) & isfinite(x1_te) & isfinite(x2_te) & isfinite(x3_te);

    x1tr = x1_tr(mtr); x2tr = x2_tr(mtr); x3tr = x3_tr(mtr); ytr = y_tr(mtr);
    x1te = x1_te(mte); x2te = x2_te(mte); x3te = x3_te(mte); yte = y_te(mte);

    for m = 1:5
        Xtr = ida_designMatrix3(m, x1tr, x2tr, x3tr);
        Xte = ida_designMatrix3(m, x1te, x2te, x3te);

        a = lsqminnorm(Xtr, ytr);

        yhat_tr = Xtr * a;
        yhat_te = Xte * a;

        [rmse_tr, r2_tr] = ida_metrics(ytr, yhat_tr);
        [rmse_te, r2_te] = ida_metrics(yte, yhat_te);

        r = r + 1;
        VnName(r) = string(cand{i});
        ModelId(r) = m;
        ModelName(r) = modelNames(m);
        p(r) = size(Xtr,2);

        RMSE_Train(r) = rmse_tr;
        R2_Train(r) = r2_tr;
        RMSE_Test(r) = rmse_te;
        R2_Test(r) = r2_te;

        if rmse_te < bestRMSE
            bestRMSE = rmse_te;
            best.VnName = string(cand{i});
            best.ModelId = m;
            best.ModelName = modelNames(m);
            best.p = size(Xtr,2);
            best.a = a;
            best.ytr = ytr; best.yhat_tr = yhat_tr;
            best.yte = yte; best.yhat_te = yhat_te;
        end
    end
end

t6 = table(VnName, ModelId, ModelName, p, RMSE_Train, R2_Train, RMSE_Test, R2_Test);
t6_sorted = sortrows(t6, 'RMSE_Test', 'ascend');
out.t6 = t6;
out.t6_sorted = t6_sorted;
out.best = best;
end

```

Лістинг 14. Функція пошуку найкращої комбінації вхідних змінних.

Скрипт спочатку зчитує назви стовпців таблиці та витягує з неї потрібні змінні x_1 , x_2 та y (V-5, V-13 та V-9). Далі він знаходить усі числові змінні у таблиці та формує список кандидатів V-n. Потім готує масиви для збереження результатів і проходиться циклом по кожному кандидату як по x_3 , ділить на навчальні та

тестові дані. Для кожного з 5 типів моделей формує матриці ознак для train використовуючи designMatrix3. Після цього обчислюються коефіцієнти на навчальних даних за допомогою lsqminnorm, обчислюються прогнози для train і test і через metrics рахуються RMSE та R^2 . Усі ці значення записуються у підготовлені масиви разом з назвою V-n, номером моделі та кількістю параметрів. На завершення скрипт формує таблицю з усіма результатами, сортує її за RMSE_test і виводить топ-10 найкращих комбінацій.

TOP-10 за RMSE_Test:

VnName	ModelId	ModelName	p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
"V_8"	5	"M5 mixed cubic"	13	171.49	0.98037	202.46	0.96921
"V_8"	4	"M4 cubic"	10	198.87	0.9736	220.35	0.96353
"V_8"	2	"M2 quad"	7	217.59	0.9684	227.16	0.96124
"V_8"	3	"M3 full quad"	10	200.83	0.97308	230.21	0.96019
"V_8"	1	"M1 linear"	4	266.26	0.95267	249.42	0.95327
"V_1"	5	"M5 mixed cubic"	13	502.94	0.83114	519.5	0.79729
"V_1"	3	"M3 full quad"	10	561.3	0.78968	587.74	0.74053
"V_1"	4	"M4 cubic"	10	564.67	0.78714	608.98	0.72144
"V_1"	2	"M2 quad"	7	611.09	0.75071	637.65	0.6946
"V_1"	1	"M1 linear"	4	699.21	0.67363	677.92	0.6548

Рисунок 16. Результати пошуку найкращої комбінації вхідних змінних.

Отже таблиця на рис. 6 показує що найкращою моделлю є Модель 5 – змішана кубічна з взаємодіями вищого порядку з V-8 як x^3 . V-8 – це ціна за квадратний метр житла на початку будівництва. Дана модель демонструє надзвичайну точність. Варто перевірити кореляцію між V-8 та V-9. І кореляція між ними дійсно сильна, адже вона дорівнює 0.9764. Що не є дивним адже фактична ціна продажу будівлі/квартири більшою мірою залежить від ціни житлової площі.

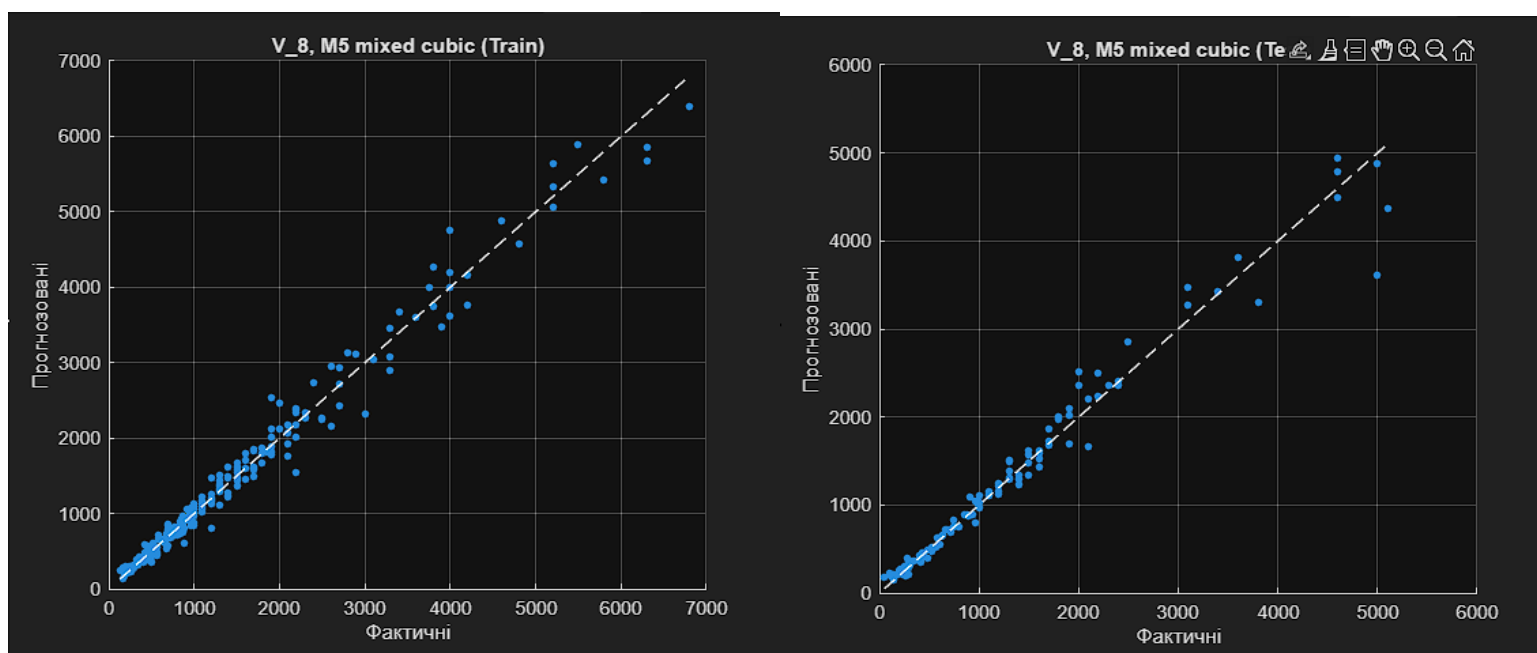


Рисунок 17. Візуалізація прогнозів M5 з V-8.

На train точки майже лежать на діагоналі, тобто модель добре відтворює фактичні значення, особливо для малих і середніх цін. На великих значеннях видно більший розкид: місцями модель недооцінює або переоцінює. На test картина схожа, але точок менше і відхилення на високих цінах помітніші, проте загалом точки також близько до діагоналі, тому модель узагальнює нормально.

Модель	$RMSE_{test}$	R^2_{test}
M2 (завдання 5)	715.49	0.61548
M5V8	202.46	0.96921
M2V8	227.16	0.96124

Таблиця 1. Порівняння моделей M2, M2V8 та M5V8.

В порівнянні з попередньою оптимальною моделлю, M5 з V-8 (M5V8) показує в 3.5 рази кращі результати RMSE та в 1.5 рази кращий R^2 . Але вона не стає новою оптимальною моделлю через її складність. Вона має 13 параметрів і важко інтерпретується. Цієї проблеми не має квадратична модель M2V8, вона має лише 7 параметрів та не значно поступається M5V8 у точності.

З вище перелічених причин **новою оптимальною моделлю стає M2V8**.

Завдання 7

Розробити лінійну регресійну модель з усіма кількісними вхідними змінними та порівняти з іншими моделями

```
if ~exist('T','var') || ~exist('trainIdx','var')
    ida_ll_data;
end

t7 = ida_fit_linear_all_numeric(T, 'V_9', trainIdx, testIdx);
disp(t7.summary);

ida_plot_yhat(t7.ytr, t7.yhat_tr, 'Лінійна (усі кількісні) (Train)');
ida_plot_yhat(t7.yte, t7.yhat_te, 'Лінійна (усі кількісні) (Test)');
```

Лістинг 15. Розробка лінійної моделі з усіма кількісними змінними.

(для ida_plot_yhat див. лістинг 12)

```
function out = ida_fit_linear_all_numeric(T, yName, trainIdx, testIdx)
vars = T.Properties.VariableNames;

y = T(:, yName);
y = y(:);

isNum = varfun(@isnumeric, T, 'OutputFormat','uniform');
predNames = vars(isNum);
predNames = setdiff(predNames, {yName}, 'stable');

Xall = double(T(:, predNames));

Xtr0 = Xall(trainIdx, :);
```

```

Xtr0 = Xall(testIdx, :);
ytr0 = y(trainIdx); ytr0 = ytr0(:);
yte0 = y(testIdx); yte0 = yte0(:);

mtr = ida_mask_finite(ytr0, Xtr0);
mte = ida_mask_finite(yte0, Xte0);

Xtr = Xtr0(mtr, :);
ytr = ytr0(mtr);
Xte = Xte0(mte, :);
yte = yte0(mte);

s = std(Xtr, 0, 1);
keep = isfinite(s) & (s > 0);

Xtr = Xtr(:, keep);
Xte = Xte(:, keep);
predKept = predNames(keep);

XtrD = [ones(size(Xtr,1),1), Xtr];
XteD = [ones(size(Xte,1),1), Xte];

a = lsqminnorm(XtrD, ytr);

yhat_tr = XtrD * a;
yhat_te = XteD * a;

[rmse_tr, r2_tr] = ida_metrics(ytr, yhat_tr);
[rmse_te, r2_te] = ida_metrics(yte, yhat_te);

out.ytr = ytr;
out.yte = yte;
out.yhat_tr = yhat_tr;
out.yhat_te = yhat_te;
out.a = a;
out.predNames = predKept;
out.p = size(XtrD,2);
out.rmse_tr = rmse_tr;
out.r2_tr = r2_tr;
out.rmse_te = rmse_te;
out.r2_te = r2_te;
out.summary = table(out.p, rmse_tr, r2_tr, rmse_te, r2_te, ...
    'VariableNames', {'p', 'RMSE_Train', 'R2_Train', 'RMSE_Test', 'R2_Test'});
end

```

Лістинг 16. Функція розробки лінійної моделі з усіма кількісними змінними.

Ця функція будує лінійну регресійну модель для вихідної змінної, яку ти задаєш через `yName`, використовуючи всі кількісні стовпці таблиці `T` як вхідні, але виключаючи саму вихідну змінну. Вона бере `y` з таблиці, знаходить усі числові колонки, формує список предикторів без `yName` і збирає з них матрицю `X`. Потім за індексами `trainIdx` і `testIdx` розділяє дані на навчальну та тестову частини, прибирає рядки з `NaN` або `Inf` через `ida_mask_finite`, після чого додатково відкидає предиктори, які на `train` мають нульову дисперсію (тобто фактично константи, що не несуть інформації). Далі додається стовпець одиниць для вільного члена, коефіцієнти моделі обчислюються на навчальних даних методом найменших квадратів через `lsqminnorm`, після чого робляться прогнози для `train` і `test` і рахуються метрики `RMSE` та `R2` для обох вибірок. У результаті функція повертає структуру `out`, де збережені очищені `y` для `train/test`,

прогнози, коефіцієнти, список реально використаних предикторів, кількість параметрів і підсумкова таблиця з метриками.

```
function m = ida_mask_finite(y, X)
y = y(:);
m = isfinite(y) & builtin('all', isfinite(X), 2);
end
```

Лістинг 17. Функція фільтрування даних від NaN та Inf.

p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
109	132.34	0.98831	141.87	0.98488

Рисунок 18. Статистика лінійної моделі з усіма змінними (M1All).

M1All (Рис. 18) є найточнішою моделлю з усіх наявних у даній роботі до тепер, але приріст точності в складності не є пропорційним приросту точності. M1All стала дуже складною і важко інтерпретабельною. Приріст при переході від M2 до M2V8 був значним і помітним, при цьому додано було лише 4 параметри. M1All додає 102 параметри але приріст точності і близько не зрівняється з попереднім приростом.

Модель	RMSE	R^2	p
M2	715.49	0.61548	3
M2V8	227.16	0.96124	7
M1All	141.87	0.98488	109
Різниця	-85.29	+0.02364	+102

Таблиця 2. Порівняння моделей M2V8 та M1All.

Спираючись на вище наведену аргументацію **оптимальною моделлю досі залишається M2V8.**

Завдання 8

Знайти моделі інших дослідників і порівняти власні результати з результатами конкурентів. (зроблено після завдання 11). За конкурента було взято модель із дослідження A Novel Machine Learning Model for Estimation of Sale Prices of Real Estate Units за авторства Mohammad Hossein Rafiei & Hojjat Adeli 2016 року. Вони розробили гібридну модель, що поєднує Deep Belief Network (DBN) з генетичним алгоритмом для оптимізації параметрів.

Модель	RMSE	R2	p	Інтерпретованість
M1V7V8 Robust	182.53	0.97497	3	Легка
DBN-GN	208.2	0.9841	103	Важка

Таблиця 3. Порівняння з конкурентом.

За таблицею видно, що робастна лінійна модель M1V7V8 має меншу похибку і набагато меншу складність, тому її простіше застосовувати й пояснювати. Модель DBN–GN дає дуже високий R2, але вона складніша за кількістю параметрів і важка для інтерпретації, тобто фактично працює як “чорна скринька”. Отже, якщо пріоритет – зрозумілість і практичність, доцільніше обирати M1V7V8 Robust, а якщо головне – максимальна здатність пояснювати варіацію даних, тоді перевага за DBN–GN, але ціною складності й низької інтерпретованості. Але якщо важлива лише точність то модель за жадібним алгоритмом на розподілі даної роботи досягла більшої точності ніж модель конкурента на своєму розподілі.

Завдання 9

Розробити лінійні регресійні моделі для усіх пар вхідних змінних, обрати кращу та порівняти з іншими моделями.

```
if ~exist('T','var') || ~exist('trainIdx','var') || ~exist('testIdx','var')
    ida_ll_data;
end
vars = T.Properties.VariableNames;
if ismember('V_9', vars), yName = 'V_9'; else, yName = 'V-9'; end

pairs = ida_scan_linear_pairs(T, yName, trainIdx, testIdx);

disp('TOP-10 пар за RMSE_Test:');
disp(pairs.results(1:min(10,height(pairs.results)), :));
```

Лістинг 17. Пошук оптимальної пари вхідних змінних.

```
function out = ida_scan_linear_pairs(T, yName, trainIdx, testIdx)
vars = T.Properties.VariableNames;
y = T{:, yName}; y = y(:);

isNum = varfun(@isnumeric, T, 'OutputFormat','uniform');
predNames = vars(isNum);
predNames = setdiff(predNames, {yName}, 'stable');

Xall = double(T{:, predNames});
Xtr0 = Xall(trainIdx, :); Xte0 = Xall(testIdx, :);
ytr0 = y(trainIdx); ytr0 = ytr0(:);
yte0 = y(testIdx); yte0 = yte0(:);

mtr = isfinite(ytr0) & builtin('all', isfinite(Xtr0), 2);
mte = isfinite(yte0) & builtin('all', isfinite(Xte0), 2);
Xtr0 = Xtr0(mtr, :); ytr0 = ytr0(mtr);
Xte0 = Xte0(mte, :); yte0 = yte0(mte);

s = std(Xtr0, 0, 1);
keep = isfinite(s) & (s > 0);
Xtr0 = Xtr0(:, keep);
Xte0 = Xte0(:, keep);
predNames = predNames(keep);

pidx = nchoosek(1:numel(predNames), 2);
```

```

K = size(pidx,1);
Var1 = strings(K,1); Var2 = strings(K,1);
RMSE_Train = zeros(K,1); R2_Train = zeros(K,1);
RMSE_Test = zeros(K,1); R2_Test = zeros(K,1);

for k = 1:K
    i = pidx(k,1); j = pidx(k,2);
    Xtr = [ones(size(Xtr0,1),1), Xtr0(:,[i j])];
    Xte = [ones(size(Xte0,1),1), Xte0(:,[i j])];
    a = lsqminnorm(Xtr, ytr0);
    yhat_tr = Xtr * a; yhat_te = Xte * a;
    [rmse_tr, r2_tr] = ida_metrics(ytr0, yhat_tr);
    [rmse_te, r2_te] = ida_metrics(yte0, yhat_te);
    Var1(k) = string(predNames{i});
    Var2(k) = string(predNames{j});
    RMSE_Train(k) = rmse_tr; R2_Train(k) = r2_tr;
    RMSE_Test(k) = rmse_te; R2_Test(k) = r2_te;
end

results = table(Var1, Var2, repmat(3,K,1), RMSE_Train, R2_Train, RMSE_Test, R2_Test,
...
    'VariableNames',
    {'Var1', 'Var2', 'p', 'RMSE_Train', 'R2_Train', 'RMSE_Test', 'R2_Test'});
out.results = sortrows(results, 'RMSE_Test', 'ascend');
end

```

Лістинг 18. Функція пошуку оптимальної пари вхідних змінних.

Ця функція перебирає всі можливі пари числових вхідних змінних у таблиці `T` і для кожної пари будує просту лінійну регресійну модель для виходу `yName`, після чого сортує отримані моделі за точністю на тестовій вибірці. Спочатку вона бере з таблиці стовпець `y` і формує список усіх числових колонок, прибираючи з нього саму вихідну змінну. Далі з цих предикторів збирається матриця ознак `Xall` і по індексах `trainIdx/testIdx` дані розділяються на навчальну та тестову частини. Потім прибираються рядки з некоректними значеннями (`NaN/Inf`) окремо для `train` і `test`, а також викидаються предиктори, які на `train` мають нульову дисперсію (тобто фактично константи, що не дають інформації). Після підготовки даних генерується список усіх пар колонок через `nchoosek`, і в циклі для кожної пари збирається матриця регресорів з одиничним стовпцем (для вільного члена) та двома вибраними змінними. Коефіцієнти моделі знаходяться на `train` через `lsqminnorm`, після чого рахуються прогнози для `train` і `test` та обчислюються метрики `RMSE` і `R2` для обох вибірок. Усі результати записуються в таблицю (імена двох змінних, `p=3`, метрики на `train` і `test`), а наприкінці ця таблиця сортується за `RMSE_Test` у зростаючому порядку і повертається як `out.results`, щоб одразу було видно, яка пара дає найкращу якість на тесті.

TOP-10 пар за RMSE_Test:						
Var1	Var2	p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
"V_7"	"V_8"	3	261.12	0.95448	216.18	0.9649
"V_8"	"V_20_4"	3	241.84	0.96096	233.52	0.95904
"V_8"	"V_20_2"	3	256.26	0.95616	233.78	0.95895
"V_8"	"V_16_4"	3	256.28	0.95615	236.21	0.95809
"V_8"	"V_16_2"	3	253.16	0.95722	237.01	0.95781
"V_8"	"V_16"	3	261.52	0.95434	237.36	0.95768
"V_8"	"V_20_3"	3	250.72	0.95804	237.5	0.95763
"V_8"	"V_16_3"	3	252.34	0.95749	238.08	0.95743
"V_8"	"V_20_1"	3	262.12	0.95413	239.27	0.957
"V_8"	"V_10"	3	264.99	0.95313	240.55	0.95654

Рисунок 18. Топ-10 пар за точністю.

V-7 – тривалість будівництва у місяцях.

V-8 – ціна за квадратний метр житла на початку будівництва.

Найкраща парна лінійна модель зі списку використовує V7 і V8. На тестовій вибірці вона працює трохи краще за M2V8: RMSE_Test у неї 216.18, тоді як у M2V8 227.16, тобто похибка менша приблизно на 11 одиниць. Також у парної моделі трохи вище R2_Test: 0.9649 проти 0.96124. При цьому вона простіша, бо має лише 3 параметри (вільний член і два коефіцієнти), а M2V8 має 7 параметрів через квадрати. На навчальній вибірці M2V8 дає кращі значення метрик, що логічно, бо вона складніша і краще підлаштовується під train, але саме за якістю на test і за простотою найкраща парна лінійна модель виглядає вигідніше.

Модель	RMSE	R ²	p
M2V8	227.16	0.96124	7
M1V7V8	216.18	0.9649	3
Різниця	-10.98	+0.00366	-4

Таблиця 4. Порівняння моделей M2V8 та M1V7V8.

Оскільки єдиний аспект де M2V8 є кращою за M1V7V8 – це передбачення на навчальних даних, що не так релевантно, як інші показники, **M1V7V8 стає новою оптимальною моделлю.**

Завдання 10

Обрати одну із моделей та дослідити як вона зміниться за робастного регресійного аналізу. Ідентифікувати викиди і проаналізувати їх.

```
if ~exist('T','var') || ~exist('trainIdx','var') || ~exist('testIdx','var')
    ida_ll_data;
end
```

```

vars = T.Properties.VariableNames;
if ismember('V_9', vars), yName = 'V_9'; else, yName = 'V-9'; end
if ismember('V_7', vars), v7 = 'V_7'; else, v7 = 'V-7'; end
if ismember('V_8', vars), v8 = 'V_8'; else, v8 = 'V-8'; end

y = T{:, yName}; y = y(:);
x7 = T{:, v7}; x7 = x7(:);
x8 = T{:, v8}; x8 = x8(:);

x7_tr0 = x7(trainIdx); x8_tr0 = x8(trainIdx); y_tr0 = y(trainIdx);
x7_te0 = x7(testIdx); x8_te0 = x8(testIdx); y_te0 = y(testIdx);

Xtr0 = [x7_tr0, x8_tr0];
Xte0 = [x7_te0, x8_te0];

mtr = isfinite(y_tr0) & builtin('all', isfinite(Xtr0), 2);
mte = isfinite(y_te0) & builtin('all', isfinite(Xte0), 2);

Xtr = Xtr0(mtr,:); ytr = y_tr0(mtr);
Xte = Xte0(mte,:); yte = y_te0(mte);

```

Лістинг 19. Скрипт завдання 10 частина 1.

Цей фрагмент готує дані для моделі M1V7V8, визначає правильні назви колонок для V9, V7 і V8, бо в файлі вони можуть бути записані як V_9 або V-9 і так само для інших змінних. Після цього витягує з таблиці V9 та два входи x7 і x8, приводить їх до вигляду стовпців і розділяє на навчальну та тестову частини за trainIdx/testIdx. Потім формує матриці ознак Xtr0 і Xte0 з двох стовпців (V7 і V8) та відкидає рядки з NaN або Inf, щоб у train і test залишилися тільки коректні спостереження Xtr, ytr, Xte, yte.

```

mu = mean(Xtr,1);
sg = std(Xtr,0,1);
sg(~isfinite(sg) | sg==0) = 1;

XtrZ = (Xtr - mu) ./ sg;
XteZ = (Xte - mu) ./ sg;

XtrD = [ones(size(XtrZ,1),1), XtrZ];
XteD = [ones(size(XteZ,1),1), XteZ];

```

Лістинг 20. Скрипт завдання 10 частина 2.

Цей фрагмент робить стандартизацію вхідних змінних і готує матриці для регресії. Спочатку обчислюються середні значення mu і стандартні відхилення sg для кожного стовпця Xtr на навчальній вибірці. Далі sg перевіряється: якщо деь стандартне відхилення нечислове або дорівнює нулю, його замінюють на 1, щоб уникнути ділення на нуль. Після цього формується стандартизований train XtrZ і test XteZ, причому для тесту використовуються ті самі mu і sg, що й для train, щоб масштабування було однаковим. Наприкінці до обох матриць додається перший стовпець з одиниць, щоб модель могла оцінити вільний член (a0).

```

b_ols = lsqminnorm(XtrD, ytr);
yhat_tr_ols = XtrD * b_ols;
yhat_te_ols = XteD * b_ols;
[b_rob, stats] = ida_robust_bisquare_irls(XtrD, ytr, 4.685, 500, 1e-6);
yhat_tr_rob = XtrD * b_rob;
yhat_te_rob = XteD * b_rob;

```



```
coef = ["a0"; "a1(V7)"; "a2(V8)"];
disp(table(coef, b_ols, b_rob, b_rob - b_ols, ...
    'VariableNames', {'Coef', 'OLS', 'Robust', 'Delta'}));
```

Лістинг 21. Скрипт завдання 10 частина 3.

У цій частині будуються дві версії однієї і тієї ж лінійної моделі та порівнюються їхні коефіцієнти. Спочатку обчислюються коефіцієнти звичайної регресії методом найменших квадратів через `lsqminnorm` на навчальних даних `XtrD` і `ytr`, після чого за цими коефіцієнтами рахуються прогнозовані значення для навчальної та тестової вибірок множенням матриці ознак на вектор коефіцієнтів. Далі оцінюються робастні коефіцієнти за допомогою ітеративно-зваженого МНК з `bisquare`-вагами (IRLS)(для `ida_robust_bisquare_irls` див. Лістинг), що зменшує вплив викидів; для цієї моделі так само будуються прогнози на `train` і `test`. Наприкінці формується таблиця з назвами коефіцієнтів `a0`, `a1` і `a2` та виводяться їх значення для OLS і Robust разом з різницею між ними, щоб було видно, як саме змінилася модель після робастного аналізу.

Coef	OLS	Robust	Delta
"a0"	1422.9	1435	12.185
"a1(V7)"	57.954	57.116	-0.83822
"a2(V8)"	1194.4	1292.9	98.54

Рисунок 19. Коефіцієнти для M1V7V8 (+Robust).

Цей вивід показує, що після робастного аналізу модель змінилась не радикально, але помітно по двох параметрах. Коефіцієнт при `V7` майже не змінився, він зменшився приблизно на 0.84, тобто вплив `V7` на прогноз залишився практично таким самим. Вільний член `a0` теж змінився мало, приблизно на +12, це невеликий зсув рівня моделі. Найбільша зміна відбулась у коефіцієнті при `V8`: він зріс приблизно на 98.5. Це означає, що робастна модель в середньому “підсилила” роль `V8` у прогнозі і стала сильніше прив’язувати ціну до `V8`, ніж звичайна OLS. Така ситуація типова, коли в даних є викиди, які тягнули OLS оцінку вниз або вбік саме по напрямку `V8`; робастна регресія зменшила їхній вплив і коефіцієнт при `V8` наблизився до тренду основної маси спостережень.

```
w = stats.w(:);
e = ytr - yhat_tr_rob;
s = stats.s;

H = XtrD * ((XtrD' * XtrD) \ XtrD');
h = diag(H);
r = e ./ (s * sqrt(1 - h));

isOut = (w < 0.5) | (abs(r) > 3);
```

Лістинг 22. Скрипт завдання 10 частина 4.

Ця частина знаходить викиди на навчальній вибірці для робастної моделі. w – це робастні ваги точок: якщо вага мала, алгоритм майже ігнорує це спостереження. e – залишки, s – робастна оцінка масштабу шуму в залишках. Далі через матрицю проєкції рахується leverage h , щоб врахувати впливовість точок за їх X . Після цього обчислюється стандартизований залишок r , тобто помилка в одиницях шуму з поправкою на leverage. Викидами `isOut` позначаються точки, у яких вага < 0.5 або $|r| > 3$.

```
figure;
scatter(ytr(~isOut), yhat_tr_rob(~isOut), 16, 'filled'); hold on;
scatter(ytr(isOut), yhat_tr_rob(isOut), 26, 'filled');

mn = min([ytr; yhat_tr_rob]); mx = max([ytr; yhat_tr_rob]);
plot([mn mx], [mn mx], '--', 'LineWidth', 1, 'Color', 'w');

grid on;
xlabel('Фактичні (Train)', 'Interpreter','none');
ylabel('Прогнозовані (Robust)', 'Interpreter','none');
title('M1(V7,V8): normal vs outliers (Train)', 'Interpreter','none');
legend('Normal','Outliers','Location','best', 'Interpreter','none');
hold off;
```

Лістинг 23. Виведення графіка для завдання 10.

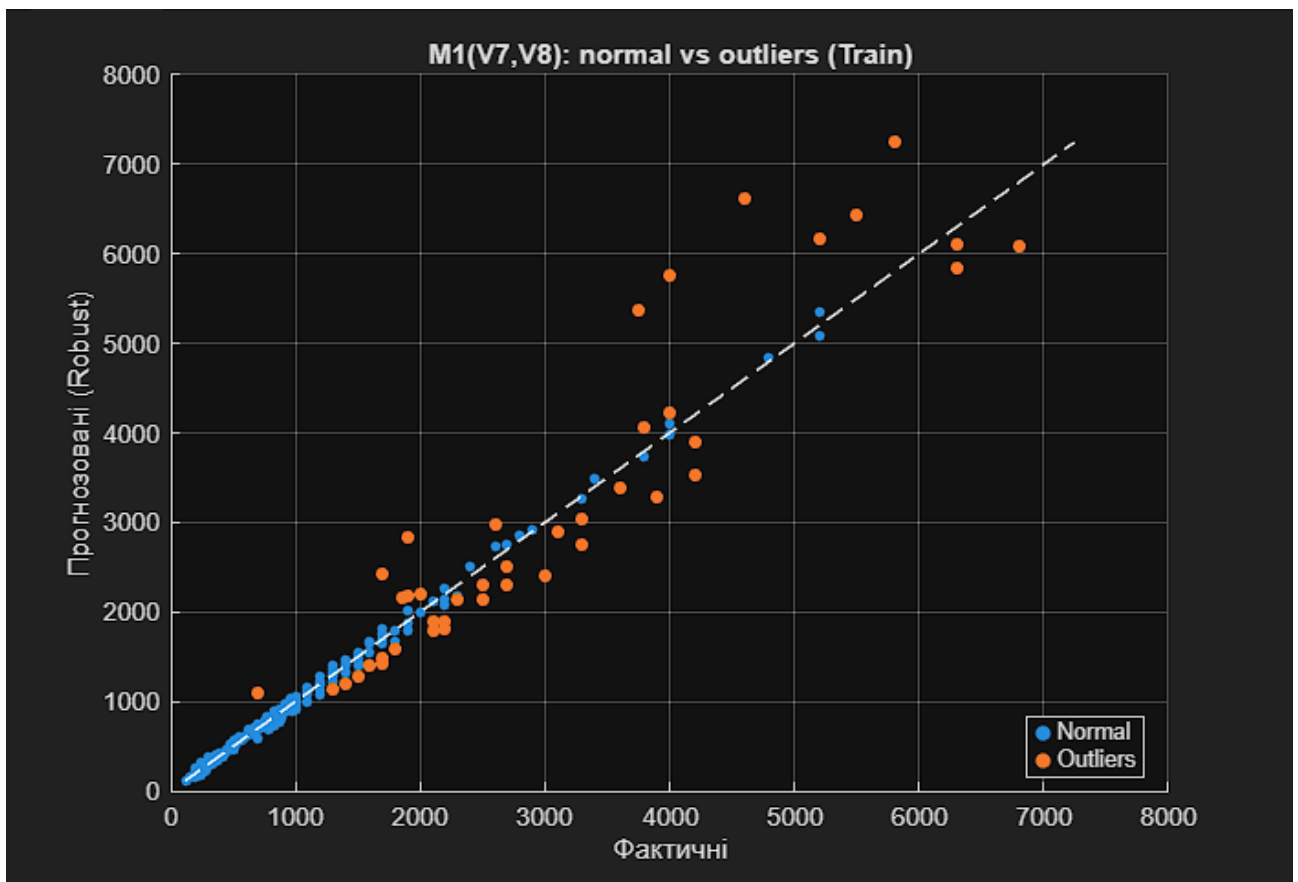


Рисунок 20. Візуалізація викидів передбачень.

Графік показує відповідність між фактичними значеннями V_9 і прогнозом робастної моделі $M1V7V8$ для навчальної вибірки. Сині точки позначають нормальні результати. Вони лежать дуже близько до діагоналі, тобто для

більшості об'єктів модель прогнозує ціну правильно або з невеликою похибкою. Помаранчеві точки позначають викиди. Вони помітно відхиляються від діагоналі – це спостереження, де модель дає великий промах, або які робастний алгоритм визначив як такі, що не відповідають загальному тренду. Їхнє відхилення вгору означає завищення прогнозу, а вниз – заниження.

Найбільші відхилення зосереджені у зоні великих значень, тобто саме в дорогому сегменті трапляються точки, які ламають звичайну регресію, і тому робастний підхід їх відокремлює та зменшує їхній вплив на модель.

```
function [b, stats] = ida_robust_bisquare_irls(X, y, tune, maxIter, tol)
    if nargin < 3 || isempty(tune), tune = 4.685; end
    if nargin < 4 || isempty(maxIter), maxIter = 200; end
    if nargin < 5 || isempty(tol), tol = 1e-6; end

    y = y(:);
    b = X \ y;

    for it = 1:maxIter
        r = y - X*b;

        s = median(abs(r)) / 0.6745;
        if ~isfinite(s) || s <= eps
            s = sqrt(mean(r.^2));
            if ~isfinite(s) || s <= eps, s = 1; end
        end

        u = r / (tune * s);
        w = (abs(u) < 1) .* (1 - u.^2).^2;

        sw = sqrt(w);
        Xw = X .* sw;
        yw = y .* sw;

        b_new = Xw \ yw;

        if norm(b_new - b) <= tol * max(1, norm(b))
            b = b_new;
            break;
        end

        b = b_new;
    end

    r = y - X*b;
    s = median(abs(r)) / 0.6745;
    if ~isfinite(s) || s <= eps
        s = sqrt(mean(r.^2));
        if ~isfinite(s) || s <= eps, s = 1; end
    end

    u = r / (tune * s);
    w = (abs(u) < 1) .* (1 - u.^2).^2;

    stats.w = w(:);
    stats.s = s;
    stats.iter = it;
end
```

Лістинг 24. Функція обрахунку робастних коефіцієнтів та статистики.

Ця функція рахує робастні коефіцієнти лінійної регресії методом IRLS з bisquare вагами. Вона стартує зі звичайного МНК $b = X \backslash y$, далі багато разів перераховує залишки, оцінює типовий масштаб помилки s , обчислює ваги w так, щоб точки з великими відхиленнями отримували дуже малі ваги аж до 0, і знову розв'язує зважену МНК задачу. Цикл зупиняється, коли коефіцієнти майже перестають змінюватись. На виході повертає робастні коефіцієнти b і статистику: ваги w , масштаб s і кількість ітерацій.

```
[rmse_tr_ols, r2_tr_ols] = ida_metrics(ytr, yhat_tr_ols);
[rmse_te_ols, r2_te_ols] = ida_metrics(yte, yhat_te_ols);

[rmse_tr_rob, r2_tr_rob] = ida_metrics(ytr, yhat_tr_rob);
[rmse_te_rob, r2_te_rob] = ida_metrics(yte, yhat_te_rob);

summary = table( ...
    ["OLS"; "Robust"], ...
    repmat(size(XtrD,2),2,1), ...
    [rmse_tr_ols; rmse_tr_rob], ...
    [r2_tr_ols; r2_tr_rob], ...
    [rmse_te_ols; rmse_te_rob], ...
    [r2_te_ols; r2_te_rob], ...
    'VariableNames', {'Type','p','RMSE_Train','R2_Train','RMSE_Test','R2_Test'} );
disp(summary);

fprintf('Outliers (train): %d of %d (%.2f%%)\n', sum(isOut), numel(isOut),
100*mean(isOut));
```

Лістине 25. Виведення статистик звичайної та робастної моделей.

Дана частина коду обраховує та виводить статистику для обох звичайної та робастної моделей.

Type	p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
OLS	3	261.12	0.95448	216.18	0.9649
Robust	3	279.29	0.94793	182.53	0.97497

Outliers (train): 45 of 260 (17.31%)

Рисунок 21. Статистика моделей.

Висновок

Робастна версія моделі M1V7V8 змінила поведінку моделі на даних з аномаліями. На навчальній вибірці якість трохи погіршилась, зате на тестовій вибірці стала кращою. Це означає, що робастний підхід менше підганяється під проблемні точки в train і дає краще узагальнення на test. Основна зміна відбулась через V8, її коефіцієнт виріс на 98.54, тоді як коефіцієнт V7 майже не змінився, а a_0 змістився незначно. Тобто викиди найбільше спотворювали оцінку впливу V8 у звичайній моделі. За оцінкою викидів робастна модель позначила 45 із 260 точок у train – 17.31% як проблемні.

Завдання 11

Дослідити залежність RMSE від кількості вхідних змінних моделі. Складність моделі нарощувати за жадібним алгоритмом від меншого значення до більшого. Обрати кращу та порівняти з іншими моделями. Додатково реалізувати жадібний алгоритм за зворотною схемою (шляхом вилучення найгіршої змінної), дослідити залежність RMSE від кількості вхідних змінних моделі та порівняти з іншими моделями.

```
vars = T.Properties.VariableNames;
if ismember('V_9', vars), yName = 'V_9'; else, yName = 'V-9'; end

y = T{:, yName}; y = y(:);

isNum = varfun(@isnumeric, T, 'OutputFormat','uniform');
predNames = vars(isNum);
predNames = setdiff(predNames, {yName}, 'stable');

Xall = double(T{:, predNames});

Xtr0 = Xall(trainIdx, :); ytr0 = y(trainIdx); ytr0 = ytr0(:);
Xte0 = Xall(testIdx, :); yte0 = y(testIdx); yte0 = yte0(:);

mtr = isfinite(ytr0) & builtin('all', isfinite(Xtr0), 2);
mte = isfinite(yte0) & builtin('all', isfinite(Xte0), 2);

Xtr0 = Xtr0(mtr,:); ytr0 = ytr0(mtr);
Xte0 = Xte0(mte,:); yte0 = yte0(mte);

s = std(Xtr0, 0, 1);
keep = isfinite(s) & (s > 0);
Xtr0 = Xtr0(:, keep);
Xte0 = Xte0(:, keep);
predNames = predNames(keep);

fwd = ida_greedy_forward_rmse(Xtr0, ytr0, Xte0, yte0, predNames);
bwd = ida_greedy_backward_rmse(Xtr0, ytr0, Xte0, yte0, predNames);

disp('FORWARD (кроки):');
disp(fwd.steps);

disp('BACKWARD (кроки):');
disp(bwd.steps);

[bestF, kF] = min(fwd.steps.RMSE_Test);
[bestB, kB] = min(bwd.steps.RMSE_Test);

fprintf('Best FORWARD: k=%d, RMSE_Test=%.4f, R2_Test=%.4f\n', kF, bestF, fwd.steps.R2_Test(kF));
fprintf('Best BACKWARD: k=%d, RMSE_Test=%.4f, R2_Test=%.4f\n', kB, bestB, bwd.steps.R2_Test(kB));

figure;
plot(fwd.steps.k, fwd.steps.RMSE_Test, '-o'); hold on;
plot(bwd.steps.k, bwd.steps.RMSE_Test, '-o');
grid on;
xlabel('Кількість змінних k', 'Interpreter','none');
ylabel('RMSE_Test', 'Interpreter','none');
title('RMSE_Test vs k (Forward vs Backward greedy)', 'Interpreter','none');
legend('Forward','Backward','Location','best', 'Interpreter','none');
hold off;
```

Цей код готує дані для жадібного відбору змінних і будує залежність RMSE від кількості входів. Він знаходить вихідну змінну V9, бере всі числові колонки як кандидати у предиктори крім V9, формує матрицю X і ділить її на train/test за trainIdx/testIdx. Потім видаляє рядки з NaN/Inf і прибирає предиктори з нульовою або некоректною дисперсією. Далі запускає два алгоритми: forward додає змінні по одній так, щоб на кожному кроці мінімізувати RMSE_Test, а backward стартує з усіх змінних і по одній викидає найгіршу, теж орієнтуючись на RMSE_Test. Після цього він виводить таблиці кроків, знаходить найкращий крок (мінімальний RMSE_Test) для кожного методу і будує один графік RMSE_Test проти k для forward і backward.

```
function out = ida_greedy_forward_rmse(Xtr0, ytr0, Xte0, yte0, predNames)
p = size(Xtr0,2);
selected = false(1,p);

kVec = (1:p)';
AddedVar = strings(p,1);
RMSE_Train = zeros(p,1); R2_Train = zeros(p,1);
RMSE_Test = zeros(p,1); R2_Test = zeros(p,1);

for k = 1:p
    bestRmse = inf;
    bestJ = 0;
    bestStats = [NaN NaN NaN NaN];

    for j = 1:p
        if selected(j), continue; end
        idx = find(selected); idx = [idx, j];

        Xtr = [ones(size(Xtr0,1),1), Xtr0(:,idx)];
        Xte = [ones(size(Xte0,1),1), Xte0(:,idx)];

        a = lsqminnorm(Xtr, ytr0);
        yhat_tr = Xtr * a;
        yhat_te = Xte * a;

        [rmse_tr, r2_tr] = ida_metrics(ytr0, yhat_tr);
        [rmse_te, r2_te] = ida_metrics(yte0, yhat_te);

        if rmse_te < bestRmse
            bestRmse = rmse_te;
            bestJ = j;
            bestStats = [rmse_tr, r2_tr, rmse_te, r2_te];
        end
    end

    selected(bestJ) = true;
    AddedVar(k) = string(predNames{bestJ});

    RMSE_Train(k) = bestStats(1);
    R2_Train(k) = bestStats(2);
    RMSE_Test(k) = bestStats(3);
    R2_Test(k) = bestStats(4);
end

pCol = 1 + kVec;
```

```

out.order = AddedVar;
out.steps = table(kVec, AddedVar, pCol, RMSE_Train, R2_Train, RMSE_Test, R2_Test, ...
    'VariableNames',
    {'k', 'AddedVar', 'p', 'RMSE_Train', 'R2_Train', 'RMSE_Test', 'R2_Test'});
end

```

Лістинг 27. Жадібний алгоритм.

Ця функція реалізує прямий жадібний відбір ознак для лінійної регресії і показує, як змінюється якість моделі, коли ми крок за кроком додаємо по одній змінній. На вході вона отримує матриці ознак для train і test (X_{tr0} , X_{te0}), цільові значення (y_{tr0} , y_{te0}) і назви змінних `predNames`. Спочатку задається кількість доступних ознак `p` і маска `selected`, яка відмічає, які змінні вже вибрані. Далі цикл $k = 1:p$: на кожному кроці перебираються всі ще не вибрані змінні j , тимчасово додається j до вже вибраних, формується матриця регресії з одиничним стовпцем (щоб був вільний член) і розв'язується задача найменших квадратів через `lsqminnorm`. Після цього рахуються прогнози на train і test та метрики RMSE і R^2 (через `ida_metrics`). З усіх кандидатів на цьому кроці вибирається той, який дає найменший RMSE на тестовій вибірці, і саме він фіксується як додана змінна для кроку k . Паралельно запам'ятовуються назва доданої змінної, RMSE/ R^2 для train і test. В кінці формується таблиця `steps`: k – скільки змінних вже у моделі, `AddedVar` – яку змінну додали на цьому кроці, p – кількість параметрів моделі, і відповідні RMSE та R^2 . Так ти отримуєш порядок додавання змінних і залежність похибки від кількості входів при жадібному нарощуванні складності.

```

function out = ida_greedy_backward_rmse(Xtr0, ytr0, Xte0, yte0, predNames)
p0 = size(Xtr0,2);
active = true(1,p0);

kVec = (p0:-1:1)';
RemovedVar = strings(p0,1);
RMSE_Train = zeros(p0,1); R2_Train = zeros(p0,1);
RMSE_Test = zeros(p0,1); R2_Test = zeros(p0,1);

for step = 1:p0
    idxActive = find(active);
    k = numel(idxActive);

    Xtr = [ones(size(Xtr0,1),1), Xtr0(:,idxActive)];
    Xte = [ones(size(Xte0,1),1), Xte0(:,idxActive)];

    a = lsqminnorm(Xtr, ytr0);
    yhat_tr = Xtr * a;
    yhat_te = Xte * a;

    [rmse_tr, r2_tr] = ida_metrics(ytr0, yhat_tr);
    [rmse_te, r2_te] = ida_metrics(yte0, yhat_te);

    RMSE_Train(step) = rmse_tr;
    R2_Train(step) = r2_tr;
    RMSE_Test(step) = rmse_te;
    R2_Test(step) = r2_te;

    if k == 1
        RemovedVar(step) = "";
        break;
    end
end

```

```

end

bestRmse = inf;
worstJ = 0;

for j = idxActive
    idxTry = idxActive(idxActive ~= j);

    Xtr2 = [ones(size(Xtr0,1),1), Xtr0(:,idxTry)];
    Xte2 = [ones(size(Xte0,1),1), Xte0(:,idxTry)];

    a2 = lsqminnorm(Xtr2, ytr0);
    yhat_te2 = Xte2 * a2;

    [rmse_te2, ~] = ida_metrics(yte0, yhat_te2);

    if rmse_te2 < bestRmse
        bestRmse = rmse_te2;
        worstJ = j;
    end
end

active(worstJ) = false;
RemovedVar(step) = string(predNames{worstJ});
end

kActual = kVec(1:step);
RemovedVar = RemovedVar(1:step);
RMSE_Train = RMSE_Train(1:step); R2_Train = R2_Train(1:step);
RMSE_Test  = RMSE_Test(1:step);  R2_Test  = R2_Test(1:step);

out.removed = RemovedVar;
out.steps = table(kActual, RemovedVar, 1 + kActual, RMSE_Train, R2_Train, RMSE_Test,
R2_Test, ...
    'VariableNames',
    {'k', 'RemovedVar', 'p', 'RMSE_Train', 'R2_Train', 'RMSE_Test', 'R2_Test'});
end

```

Лістинг 28. Зворотний жадібний алгоритм.

Функція реалізує зворотний жадібний відбір змінних для лінійної регресії. Вона стартує з моделлю, де використані всі предиктори, на кожному кроці будує регресію на поточному наборі змінних, рахує RMSE і R^2 на train і test та зберігає ці значення. Потім вона перебирає вилучення кожної активної змінної по черзі, дивиться, при якому вилученні RMSE на тесті стає найменшим, і саме цю змінну вважає найгіршою та вимикає. Так крок за кроком модель спрощується до однієї змінної, а на виході формується таблиця, де для кожної кількості змінних k записано: яку змінну прибрави, скільки параметрів стало в моделі і які метрики RMSE/ R^2 отримані на train/test.

FORWARD (кроки):						
k	AddedVar	p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
1	"V_8"	2	267.44	0.95225	249.63	0.9532
2	"V_7"	3	261.12	0.95448	216.18	0.9649
3	"V_16_4"	4	250.16	0.95822	201.73	0.96943
4	"V_26_3"	5	228.52	0.96514	181.87	0.97515
5	"V_17_3"	6	168.95	0.98095	164.57	0.97966
6	"V_29"	7	159.22	0.98308	152.71	0.98248
7	"V_17_1"	8	156.33	0.98369	149.7	0.98317
8	"V_10"	9	154.13	0.98414	146.57	0.98386
9	"V_5"	10	150.73	0.98483	133.27	0.98666
10	"COMPLETIONQUARTER"	11	149.89	0.985	130.76	0.98716
11	"V_16"	12	149.63	0.98505	128.96	0.98751
12	"V_22_4"	13	149.32	0.98512	127.88	0.98772
13	"V_21"	14	148.55	0.98527	126.65	0.98795
14	"V_14_3"	15	148.45	0.98529	126.11	0.98805
15	"COMPLETIONYEAR"	16	148.31	0.98532	125.54	0.98816
16	"V_20_4"	17	147.61	0.98545	123.79	0.98849
17	"V_25"	18	147.39	0.9855	121.79	0.98886
18	"V_26_4"	19	147.3	0.98551	121.27	0.98895
19	"V_26_1"	20	147.03	0.98557	120.49	0.98909
20	"V_4"	21	146.8	0.98561	120.05	0.98918
21	"V_29_1"	22	146.78	0.98562	119.61	0.98925

Рисунок 22. Перші 20 ітерацій жадібного алгоритму.

На рис. 22 взято перші 20 ітерацій за жадібним алгоритмом. Першими, тобто найкращими змінними були додані змінні, які складають оптимальну (якщо не брати її робастну версію) лінійну модель M1V7V8. На зображенні видно що при додаванні нових змінних модель по трохи стає точнішою і вже на 20-ій ітерації R^2 зріс на 0.02428.

```
Best FORWARD: k=47, RMSE_Test=110.3738, R2_Test=0.9908
Best BACKWARD: k=57, RMSE_Test=114.8179, R2_Test=0.9901
```

Рисунок 23. Найкращі результати для обох жадібних алгоритмів.

Зростання продовжується аж до $k = 47$ і потім починає спадати. Це робить модель із 47 змінними найкращою не лише в даній вибірці, а і загалом найточнішою моделлю в лабораторній роботі до цього етапу з показниками $RMSE = 110.3738$, $R^2 = 0.99085$.

Модель	RMSE	R^2	p
M2V8	227.16	0.96124	7
M1V7V8	216.18	0.9649	3
M1V7V8 Robust	182.53	0.97497	3
Greedy Forward k=47 (GF47)	110.3738	0.99085	48

Таблиця 5. Порівняння попередніх моделей M2V8 та M1V7V8 (+Robust) з результатом за жадібним алгоритмом.

За таблицею 5 видно, що найкращу точність показує Greedy Forward, бо саме для нього найменша похибка і найбільший коефіцієнт детермінації. Це означає, що

додавання великої кількості вхідних змінних дало суттєвий виграш у якості прогнозу порівняно з простими моделями, які використовують лише кілька ознак. Моделі на двох змінних помітно слабші за точністю. Також видно, що ускладнення у вигляді квадратичних членів у цьому випадку не допомогло: квадратична модель не дала кращого узагальнення, ніж лінійна, тобто додаткова нелінійність тут швидше зайва.

Робастна версія лінійної моделі на двох змінних дає найкращий результат серед простих моделей. Це вказує на наявність викидів у даних і на те, що робастна оцінка краще “ігнорує” аномальні точки, зменшуючи їхній вплив на коефіцієнти. Проте навіть у такому вигляді проста модель поступається Greedy Forward за точністю. За складністю Greedy Forward є найважчим варіантом: він має багато параметрів, тому гірше інтерпретується. Отже, якщо головний критерій — точність, доцільно обирати Greedy Forward, а якщо потрібна проста і пояснювана модель, найбільш практичним компромісом буде робастна лінійна модель на двох змінних. Тому **робастна M1V7V8 залишається оптимальною моделлю.**

BACKWARD (кроки):						
k	RemovedVar	p	RMSE_Train	R2_Train	RMSE_Test	R2_Test
108	"V_24_1"	109	132.34	0.98831	141.87	0.98488
107	"V_1"	108	132.34	0.98831	128.42	0.98761
106	"V_24_2"	107	134.08	0.988	125.77	0.98812
105	"V_13_3"	106	134.08	0.988	121.05	0.98899
104	"V_12"	105	134.08	0.988	120.04	0.98918
103	"V_15_1"	104	134.08	0.988	119.79	0.98922
102	"V_11_4"	103	134.08	0.988	119.45	0.98928
101	"V_20_2"	102	134.08	0.988	119.06	0.98935
100	"V_25_2"	101	134.08	0.988	118.76	0.98941
99	"V_7"	100	134.08	0.988	118.52	0.98945
98	"V_4"	99	134.08	0.988	118.4	0.98947
97	"V_25"	98	134.26	0.98797	118.21	0.9895
96	"V_14"	97	134.26	0.98797	117.76	0.98958
95	"V_20_1"	96	134.26	0.98797	117.62	0.98961
94	"V_14_1"	95	134.26	0.98797	117.54	0.98962
93	"V_20_4"	94	134.26	0.98797	117.5	0.98963
92	"STARTYEAR"	93	134.26	0.98797	117.46	0.98964
91	"V_14_2"	92	134.26	0.98797	117.43	0.98964
90	"V_26"	91	134.26	0.98797	117.4	0.98965
89	"V_26_4"	90	134.26	0.98797	117.36	0.98965
88	"V_12_3"	89	134.26	0.98797	117.33	0.98966

Рисунок 24. Перші 20 ітерацій зворотнього жадібного алгоритму.

На рис. 23 наведено перші 20 ітерацій зворотного жадібного алгоритму (Greedy Backward). На відміну від прямої схеми, тут робота починається з максимальної кількості кількісних змінних, а далі на кожному кроці з моделі вилучається одна змінна. Вилучається саме та, видалення якої дає найкращий результат на тестовій вибірці (тобто найменше погіршує RMSE або навіть зменшує його). На перших кроках часто відсікаються зайві або дублюючі ознаки, через що якість узагальнення може покращуватись, але після проходження оптимальної точки подальше вилучення змінних починає забирати корисну інформацію і похибка зростає.

Модель	RMSE	R^2	p
M2V8	227.16	0.96124	7
M1V7V8	216.18	0.9649	3
M1V7V8 Robust	182.53	0.97497	3
Greedy Forward k=47	110.3738	0.99085	48
Greedy Backward k=57	114.8179	0.9901	57

Таблиця 6. Порівняння попередніх моделей M2V8 та M1V7V8 (+Robust) з результатом за зворотнім жадібним алгоритмом.

Найкращий результат для Greedy Backward отримано при $k = 57$. За якістю прогнозу він дуже близький до Greedy Forward, але потребує більшої кількості змінних, тому є складнішим для інтерпретації. З практичної точки зору це означає, що зворотна схема дає конкурентний результат, але як оптимальніший варіант за співвідношенням точності та складності доцільніше залишати Greedy Forward, оскільки він досягає майже тієї ж точності з меншою кількістю ознак.

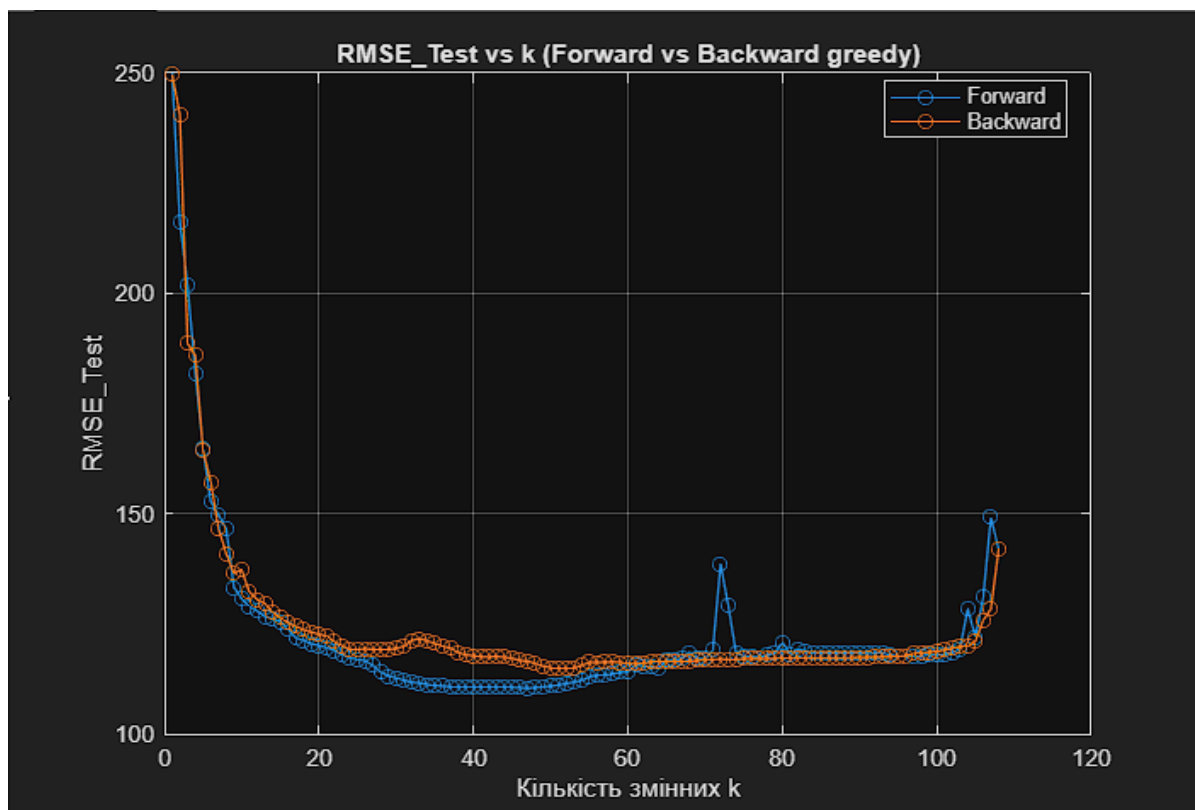


Рисунок 25. Візуалізація RMSE по ітераціях кількості змінних.

Графік показує, що найбільший виграш у точності дає додавання перших кількох змінних RMSE швидко падає, а далі крива виходить доволі рівною, де додавання або видалення чергових ознак уже мало впливає на результат. Це типова ситуація, коли після певної кількості предикторів модель уже пояснює основну частину варіації, а подальше ускладнення дає лише невелике покращення або взагалі починає шкодити.

Для Forward видно чіткий мінімум приблизно в середині діапазону k , після якого при збільшенні числа змінних RMSE поступово зростає – це ознака того, що зайві змінні починають додавати шум і погіршувати узагальнення. Для Backward форма дуже схожа, але оптимальна точка зміщена в бік більшого k , тобто зворотна схема залишає більше змінних, щоб дійти до найкращого результату.

Загалом обидві криві близькі, тому обидва підходи приводять до подібної якості, але Forward досягає свого найкращого RMSE раніше і з меншою кількістю змінних, отже виглядає практичнішим за критерієм точності та складності. Також на Forward є локальні сплески RMSE на окремих k – це зазвичай наслідок того, що на конкретному кроці в модель потрапляє невдала або сильно корельована змінна, яка робить оцінку коефіцієнтів менш стабільною.

Висновок з виконання роботи

У ході лабораторної роботи було побудовано й порівняно серію регресійних моделей для прогнозування V9 за даними Residential Building Data Set. Спочатку перевірено репрезентативність розбиття train/test і проаналізовано однофакторні залежності, після чого синтезовано базові моделі для пари V5–V13 та показано, що просте ускладнення поліномами не гарантує кращого узагальнення на тесті.

Далі досліджено розширення постановки за рахунок додаткових змінних і встановлено, що включення V8 різко покращує якість прогнозу, що узгоджується з високим зв'язком V8 із V9. Для парних лінійних моделей найкращою виявилась комбінація V7 та V8, а робастний аналіз підтвердив наявність викидів і продемонстрував, що зменшення їх впливу покращує узагальнення: коефіцієнти змінюються помірно, але помилки на тесті зменшуються, а частина спостережень отримує ознаки аномальних.

За жадібними алгоритмами показано залежність RMSE від кількості змінних: на початку додавання ознак дає найбільший виграш, далі покращення сповільнюється, а після оптимальної точки ускладнення починає погіршувати узагальнення. Forward дає найкращу точність при меншій кількості змінних, тоді

як Backward досягає дуже близької якості, але з більшою складністю, тому з практичної точки зору виглядає менш зручним.

Підсумково, за критерієм максимальної точності найкращою є модель Greedy Forward (оптимальний крок за таблицями результатів), але за співвідношенням точність–інтерпретованість більш практичним вибором залишається робастна лінійна модель на V7 та V8, оскільки вона проста, стійка до викидів і дає стабільний результат на тестовій вибірці.